KHÓA LUẬN TỐT NGHIỆP · ĐỀ XUẤT ĐỀ TÀI ĐỂ PHÊ DUYỆT

Phát hiện Tự động các Lỗ hổng Phân quyền có Trạng thái trong REST API

Mở rộng EvoMaster với một oracle kiểm soát truy cập theo trạng thái/thời gian

Sinh viên	Minh Nguyen ‹xác nhận họ tên / MSSV›	**Giảng viên hướng dẫn**	‹tên GVHD›
Ngành / Khoa	‹ngành / khoa›	**Ngày**	12/07/2026
Thời lượng	~3 tháng (kèm kế hoạch chi tiết)	**Lĩnh vực**	Kiểm thử phần mềm · Bảo mật API

Tóm tắt. Khóa luận này đề xuất một kỹ thuật tự động nhằm phát hiện một lớp lỗ hổng bảo mật của REST API mà các công cụ hiện nay chưa thể tìm ra: **lỗ hổng phân quyền có trạng thái** (*stateful authorization*) — tình huống người dùng vẫn giữ được quyền truy cập *sau khi* quyền đó lẽ ra đã bị thu hồi (ví dụ: một quản trị viên đã bị hạ quyền nhưng vẫn xóa được dữ liệu). Công trình được xây dựng trực tiếp trên **EvoMaster** — công cụ fuzzing REST API mã nguồn mở hàng đầu — và trên một mở rộng rất gần đây (tháng 4/2026) đã bổ sung các kiểm tra phân quyền ở mức *từng yêu cầu*. Đóng góp của tôi là một **oracle phân quyền theo thời gian/trạng thái (đa bước)** dựa trên *kiểm thử biến hình* (metamorphic testing), cùng cơ chế sinh chuỗi lời gọi cần thiết để kích hoạt nó, và được đánh giá trên các bộ dữ liệu chuẩn cùng các API cố tình chứa lỗ hổng. Một **phương án dự phòng rủi ro thấp (Phương án C)** — nghiên cứu thực nghiệm về hiệu quả và tính tái lập của các công cụ hiện có kèm một cải tiến có trọng điểm — cũng được đưa vào và dùng chung hạ tầng, nên đề tài chắc chắn có thể hoàn thành theo một trong hai hướng.

1. Bối cảnh: công cụ EvoMaster

EvoMaster (github.com/WebFuzzing/EvoMaster) là một công cụ mã nguồn mở, ứng dụng AI, tự động sinh các ca kiểm thử ở mức hệ thống ("fuzzing") cho các web API. Đây là nền tảng phù hợp cho khóa luận vì ba lý do: nó là công cụ tốt nhất hiện nay theo các so sánh độc lập, được viết bằng **Kotlin/Java** (đúng với nền tảng kỹ thuật của tôi), và được duy trì tích cực kèm một bộ ứng dụng thực tế để thử nghiệm.

Cách hoạt động

- **Đầu vào.** Một đặc tả OpenAPI/Swagger (v2/v3). Ở chế độ *hộp đen* chỉ cần đặc tả và một URL đang chạy; ở chế độ *hộp trắng* (chỉ JVM) nó còn phân tích mã bytecode của ứng dụng để sinh kiểm thử hiệu quả hơn nhiều.
- **Cơ chế.** Một *thuật toán tiến hóa* dựa trên tìm kiếm, tiến hóa các chuỗi yêu cầu HTTP nhằm tối đa độ bao phủ mã và khả năng phát hiện lỗi — bao gồm việc tự động *nối chuỗi* lời gọi (ví dụ `POST /users` → tái sử dụng `id` trả về trong `GET /users/{id}`).
- **Oracle (những gì nó gắn cờ là lỗi).** Mặc định: lỗi máy chủ HTTP `500`, sai lệch giữa phản hồi và đặc tả, và — từ 2026 — vi phạm chính sách truy cập.
- **Đầu ra.** Các ca kiểm thử JUnit (Java/Kotlin) chạy lại được để tái hiện từng lỗi, kèm một báo cáo HTML tương tác.

Tôi có thể xây dựng gì trên nền tảng đó

EvoMaster đã giải quyết sẵn phần hạ tầng phức tạp mà nếu không có nó khóa luận sẽ phải tự làm lại: phân tích đặc tả API, sinh chuỗi yêu cầu hợp lệ, xử lý xác thực, và xuất ra kiểm thử tái lập được. Nhờ đó tôi tập trung công sức vào *câu hỏi nghiên cứu* — một năng lực phát hiện lỗi mới — thay vì viết lại một công cụ fuzzing. Tôi mở rộng nó bằng cách thêm (a) một cách *sinh* chuỗi kiểm thử mới và (b) một *oracle* mới để phán định xem một chuỗi có làm lộ ra lỗi hay không.

2. Phương án A (chính): phát hiện lỗ hổng phân quyền có trạng thái

2.1 Vấn đề

Phân quyền (*ai được phép làm gì*) là **lớp lỗ hổng API phổ biến nhất trong thực tế** (OWASP API Security Top 10). Những lỗi nguy hiểm nhất không phải là dữ liệu đầu vào sai định dạng, mà là các **thay đổi quyền theo thời gian** mà máy chủ không thực thi đúng. Các công cụ tự động hiện nay chỉ kiểm tra phân quyền tại một thời điểm cố định, nên toàn bộ lớp lỗi này không bị phát hiện. Ba ví dụ cụ thể:

Ví dụ A1 — quyền vẫn còn sau khi bị hạ cấp vai trò

```
1. Đăng nhập với vai trò ADMIN       -> nhận token phiên T
2. Admin tạo /reports/9              -> 201 Created
3. Admin bị hạ cấp xuống VIEWER     -> [CHUYỂN TRẠNG THÁI]
4. Dùng CHÍNH token T:  DELETE /reports/9
   Kỳ vọng: 403 Forbidden  (viewer không được phép xóa)
   Thực tế: 204 No Content  -> LỖI PHÂN QUYỀN CÓ TRẠNG THÁI
```

Ví dụ A2 — quyền truy cập vẫn tồn tại sau khi chuyển quyền sở hữu

Người dùng A sở hữu một ví và chuyển quyền sở hữu cho B — nhưng A *vẫn* rút tiền được từ ví đó.

Ví dụ A3 — quyền lợi vẫn còn sau khi đã hết hạn

Một gói premium hết hạn, nhưng endpoint chỉ dành cho premium vẫn trả về `200 OK`.

Trong mọi trường hợp, lỗ hổng vô hình với một phép kiểm tra **đơn yêu cầu**: nó chỉ xuất hiện xuyên suốt một **chuỗi** có chứa một bước làm giảm quyền. Khoảng trống đó chính là mục tiêu của khóa luận.

2.2 Nền tảng chúng tôi kế thừa (phần đã có sẵn)

- **EvoMaster** — cung cấp cơ chế sinh chuỗi, xử lý xác thực, và xuất kiểm thử.
- **Şahin, Zhang & Arcuri (2026)** — "Enhancing REST API Fuzzing with Access Policy Violation Checks and Injection Attacks" (arXiv:2604.00702). Công trình này, do chính nhóm EvoMaster thực hiện, đã bổ sung chín oracle bảo mật, trong đó có các kiểm tra phân quyền *đơn yêu cầu* (so sánh phản hồi giữa các người dùng khác nhau cho *cùng một* yêu cầu). Đây là **đường cơ sở (baseline) trực tiếp** của tôi.

Điều quan trọng: cả chín oracle của bài đó đều thuộc trục *khác-người / một-thời-điểm* — không oracle nào mô hình hóa một **chuyển tiếp làm giảm quyền theo thời gian**. Phần "hướng nghiên cứu tương lai" của họ chỉ nêu chung chung ("thiết kế thêm các oracle tự động để phát hiện những loại lỗi bảo mật khác"), **không** gọi tên phân quyền có trạng thái. Đáng chú ý, ngay trong thực nghiệm (Mục 5.4) họ gặp đúng các endpoint *chỉnh sửa thông tin đăng nhập/vai trò người dùng* nhưng chỉ coi đó là một trở ngại kỹ thuật (làm token đầu phiên hết

hiệu lực), **không** biến nó thành phép thử phân quyền theo thời gian. Đó chính là chỗ khóa luận này bước vào — nền tảng vừa vững vừa mới, và bước tiếp theo này chưa ai thực hiện.

2.3 Đóng góp của chúng tôi

Tôi bổ sung **chiều thời gian / trạng thái** mà baseline còn thiếu, sử dụng **kiểm thử biến hình** (metamorphic testing) — một kỹ thuật đã được kiểm chứng, kiểm tra một tính chất dạng *"thay đổi đầu vào theo cách X thì đầu ra phải thay đổi theo cách Y."* Tính chất ở đây mang tính phổ quát và độc lập với từng API:

> **ORACLE CỐT LÕI**
> **"Việc thu hồi một quyền không bao giờ được làm tăng quyền truy cập của người dùng."** Nếu một hành động được quan sát là *được phép*, và sau đó xảy ra một bước chuyển làm giảm quyền, thì cùng hành động đó về sau phải *bị từ chối*. Nếu vẫn được phép, oracle báo cáo một lỗ hổng.

Vì oracle được định nghĩa hoàn toàn trên các *phản hồi HTTP quan sát được* (được phép → bị từ chối), nó không phụ thuộc vào cách mỗi máy chủ hiện thực phân quyền bên trong — cùng một kỹ thuật áp dụng cho mọi REST API. Cụ thể, khóa luận sẽ tạo ra:

1. **Một tập các quan hệ biến hình về phân quyền** (thu hồi quyền, chuyển quyền sở hữu, hết hạn quyền lợi).
2. **Một bộ sinh chuỗi có nhận biết bước chuyển** trong EvoMaster, tạo ra các chuỗi `cấp quyền → chuyển trạng thái → thử lại` (thao tác chuyển trạng thái được suy ra từ đặc tả và từ các tên như *revoke, demote, expire*, kèm chú thích tùy chọn ở mức nhẹ).
3. **Oracle theo thời gian** đánh giá quan hệ trên các phản hồi và xuất ra một ca kiểm thử thất bại tái lập được.
4. **Một bộ dữ liệu chuẩn nhỏ** với các lỗi phân quyền có trạng thái được cấy sẵn (cộng đồng hiện chưa có bộ này).

Khía cạnh	Baseline (Şahin và cs., 2026)	Khóa luận này (Phương án A)
Trục so sánh	Giữa các **người dùng** — một ảnh chụp tĩnh	Giữa **thời gian / trạng thái** — qua một bước chuyển
Dạng kiểm thử	Phát lại một yêu cầu dưới danh nghĩa người dùng khác	`cấp quyền → chuyển trạng thái → thử lại`
Loại oracle	Vi sai (người dùng A vs B, ngay lúc này)	Biến hình (trước vs sau một thay đổi)
Phát hiện A1–A3?	Không (mù về mặt cấu trúc)	Có (lớp mục tiêu)

> **DIỄN ĐẠT MỘT CÂU**
> Công trình trước kiểm tra xem hiện tại *kẻ sai người* có đang giữ chìa khóa hay không. Khóa luận này kiểm tra xem một chiếc chìa khóa có *còn mở được cửa sau khi lẽ ra đã bị thu hồi* hay không — tức là phân quyền khi nó thay đổi theo một chuỗi, điều mà chưa công cụ fuzzing REST hộp đen nào hiện kiểm tra.

2.4 Kế hoạch đánh giá

Khả năng phát hiện được đo trên các API cố tình chứa lỗ hổng; tỉ lệ dương tính giả được đo trên các API thực, đúng đắn. Ba tầng mục tiêu, đều chạy cục bộ bằng Docker:

Tầng	Mục tiêu	Vai trò trong đánh giá
1	Một API **Spring Boot** nhỏ do tôi tự xây, có cấy sẵn một lỗi hạ cấp vai trò	Chân lý nền có kiểm soát; đảm bảo có một ca dương tính tiếp cận được
2	**VAmPI** & **crAPI** (các API lỗ hổng của OWASP)	Mục tiêu phát hiện chuẩn cộng đồng (độ nhạy / recall)
3	**WFD** — Web Fuzzing Dataset (~45 ứng dụng thực)	Tính thực tế & tỉ lệ dương tính giả (độ chính xác)

So sánh trực tiếp: bật các oracle của baseline và chạy cả hai công cụ trên cùng mục tiêu; thành công = công cụ của tôi tìm ra các lỗi *có trạng thái* mà baseline không thể, với tỉ lệ dương tính giả chấp nhận được. Các chỉ số chính: số lỗi phát hiện, độ chính xác / tỉ lệ dương tính giả, và độ bao phủ mã/endpoint.

2.5 Tính khả thi & rủi ro

Độ khó: trung bình — thử thách vừa phải, không quá sức. Phần lớn đã được giảm rủi ro sẵn (mục tiêu chạy bằng Docker, cơ chế sinh chuỗi có sẵn của EvoMaster, mã nguồn Kotlin, và một bài báo baseline trực tiếp làm khuôn mẫu). Các thách thức nghiên cứu thực sự — cũng là phần cốt lõi của khóa luận — là **kiểm soát dương tính giả** và **khả năng tiếp cận** (đưa bộ fuzzing đi đủ sâu để chạm tới trạng thái chứa lỗ hổng).

> RỦI RO CHÍNH & CÁCH GIẢM THIỂU
> *Khả năng tiếp cận* — các API thực có thể quá sâu để bộ fuzzing chạm tới trạng thái lỗ hổng, làm giảm khả năng phát hiện ở Tầng 3. **Giảm thiểu:** ứng dụng có kiểm soát ở Tầng 1 và các lỗi được cấy ở Tầng 2 đảm bảo luôn có ca dương tính tiếp cận được, nên kỹ thuật luôn có thể được minh chứng; kết quả Tầng 3 sau đó được báo cáo trung thực kèm giới hạn — đúng như một chương đánh giá cần có.

3. Phương án C (dự phòng): nghiên cứu thực nghiệm + một cải tiến trọng điểm

Nếu oracle theo thời gian khó hơn dự kiến, đề tài **suy giảm một cách êm đẹp** thành một **nghiên cứu thực nghiệm** dùng lại *chính* hạ tầng đó (EvoMaster + WFD + VAmPI/crAPI) — nên không lãng phí công sức. Hai hướng khả thi:

- **C1 — Tính tái lập / độ bất định (flakiness).** Chạy EvoMaster trên ~10 ứng dụng WFD, lặp lại nhiều lần, đo xem bao nhiêu ca kiểm thử sinh ra bị *bất định* (lần này đạt, lần sau trượt — ví dụ do kiểm tra trên `id` tự tăng hoặc dấu thời gian), xây dựng phân loại nguyên nhân, và hiện thực một biện pháp giảm thiểu, cho thấy tỉ lệ bất định giảm. Đây là chủ đề nghiên cứu còn nóng năm 2026 — Zhang, Shen, Arcuri & Yue, arXiv:2603.28452.
- **C2 — Hiệu quả của các oracle bảo mật.** Đo mức độ các công cụ hiện có phát hiện từng hạng mục OWASP API trên VAmPI/crAPI, xác định hạng mục yếu nhất, và thêm một cải tiến. Nếu hạng mục yếu nhất là phân quyền có trạng thái, điều này *chứng minh bằng thực nghiệm* sự cần thiết của Phương án A.

Vì sao đây là phương án dự phòng an toàn: kết quả thực nghiệm chắc chắn tồn tại bất kể "bản sửa" hoạt động ra sao, nên việc hoàn thành được đảm bảo. Một mốc quyết định ở cuối Tuần 5 (bên dưới) sẽ chọn giữa tiếp tục A hay chuyển sang C, dựa trên dữ liệu đã thu thập.

4. Lộ trình dự kiến (~12 tuần)

Tuần	Cột mốc
1–2	Thiết lập môi trường; tái lập EvoMaster trên WFD/VAmPI/crAPI với các oracle bảo mật baseline được bật; xây API Spring Boot Tầng 1 có cấy lỗi.
3–4	Hoàn thiện tổng quan tài liệu (xác nhận khoảng trống vẫn còn); hình thức hóa các quan hệ biến hình về phân quyền.
5	**Mốc quyết định:** tiếp tục Phương án A, hoặc chuyển sang Phương án C dựa trên dữ liệu đã thu thập.
5–7	Hiện thực bộ sinh chuỗi có nhận biết bước chuyển và oracle theo thời gian trong EvoMaster (Kotlin).
8–9	Cấy lỗi vào bộ dữ liệu; chạy thực nghiệm; so sánh trực tiếp với baseline.
10–11	Phân tích (phát hiện, dương tính giả); tinh chỉnh để tăng độ chính xác.
12	Viết báo cáo, phần giới hạn, và chuẩn bị bảo vệ.

5. Tài liệu tham khảo & nguồn lực chính

1. A. Arcuri và cs. *EvoMaster: search-based fuzzing for REST, GraphQL and RPC APIs.* Công cụ mã nguồn mở — github.com/WebFuzzing/EvoMaster. Báo cáo công cụ: *Automated Software Engineering*, 2024, doi:10.1007/s10515-024-00478-1.
2. Ö. Şahin, M. Zhang, A. Arcuri. *Enhancing REST API Fuzzing with Access Policy Violation Checks and Injection Attacks.* 2026 — arXiv:2604.00702. **(Đường cơ sở trực tiếp.)**
3. M. Zhang, C. Shen, A. Arcuri, T. Yue. *Detecting and Mitigating Flakiness in REST API Fuzzing.* 2026 — arXiv:2603.28452. (Nền tảng cho phương án dự phòng C1.)
4. M. Kim, Q. Xin, S. Sinha, A. Orso. *Automated Test Generation for REST APIs: No Time to Rest Yet.* ISSTA 2022 — doi:10.1145/3533767.3534401.
5. *Web Fuzzing Dataset (WFD)* — bộ dữ liệu chuẩn các web API thực: github.com/WebFuzzing/Dataset.
6. *VAmPI* — REST API chứa lỗ hổng: github.com/erev0s/VAmPI. · *crAPI* (OWASP): github.com/OWASP/crAPI.
7. OWASP API Security Top 10 (2023) — API1: Broken Object-Level Authorization; API5: Broken Function-Level Authorization.